Big Data & BD NoSQL

Dr FERRAHI ibtisam
Maitre de conférences B, USTHB

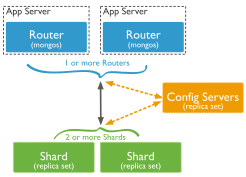
Architecture de mongoDB

Architecture Sharding

- MongoDB utilise la notion de sharding (horizontal scaling ou scale out) pour stocker ses données dans le cluster
  - Division des données et leurs distributions entre plusieurs servers ou shards
  - Chaque shard est une base indépendante et mis ensemble, forment une seule base de données logique
- Réduction du nombre d'opération que chaque machine gère
  - Chaque machine gère les données qui y sont stockées
- Réduction de la quantité de données que le serveur a besoin de stocker
  - Plus le cluster grandit moins un serveur contient des données

Architecture Sharding

- **Shards**
  - Stockent les données
  - Les données sont distribuées et répliquées sur les shards
- **Query routers**
  - Instance mongo
  - Interfaçage avec les applications client
  - Redirige les opérations vers les shard
  - Plusieurs query router pour la répartition des tâches
- **Config servers**
  - Stockent les métas données du cluster
  - Définissent le mappage entre les data et les shards
  - Dans un environnement de production 3 config servers doivent être définis



Instruction MongoDB Sharding

- Lancer les Config Servers
  - mongod --configsvr --replSet configReplSet --dbpath /data/configdb --port 27019
- Lancer les Shards
  - mongod --shardsvr --replSet shardReplSet1 --dbpath /data/shard1 --port 27018
- Lancer le Query Router
  - mongos --configdb configReplSet/config1:27019,config2:27019,config3:27019
- Ajouter un Shard au cluster
  - sh.addShard("shardReplSet1/shard1:27018,shard2:27018,shard3:27018")

- Activer le sharding sur une base
  - **sh.enableSharding("ma_base")**
  - **sh.shardCollection("ma_base.ma_collection", { shardKey: 1 })**

Partitionnement des données

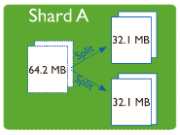
- Partitionnement des données au niveau des collections via la *shard key*
  - Champs simple ou composé indexé qui existe dans chaque documents de la collection
- MongoDB divise les valeurs de la clef en morceaux (chunks) et les distribuent de manière équitable entre les shards
- La répartition des clefs suit l'une de ces méthodes de partitionnement
  - Partitionnement basé sur rang → méthode ordonnée
  - Partitionnement basé sur le hash → méthode aléatoire
  - Partitionnement basé sur les tags

Maintien d'une distribution équitable

- **L'ajout de nouvelles données ou nouveau serveurs peut rendre la distribution des données déséquilibrée**
  - **Splitting**
  - **Balancer**

Maintien d'une distribution équitable

- **L'ajout de nouvelles données ou nouveau serveurs peut rendre la distribution des données déséquilibrée**
- **Splitting:**
  - Evite d'avoir **des chunks trop large**
  - Quand la taille d'un chunks augmente au-delà d'une valeur prédéfinie (chunk size), MongoDB devise cet ensemble de données en deux sur le même shard
  - Les insertions et les modifications déclenchent les splits
  - Un split change les méta-données, mais ne fait pas migrer les données ni n'affecte le contenu des shards

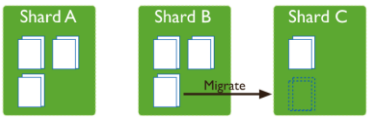


Maintien d'une distribution équitable

- **Balancing**
- **Balacer:** processus en arrière plan, gérant les migrations de **chunks**
  - Peut être lancé à partir de n'importe quel **Query Router**
  - Quand la distribution des données est déséquilibrée, le balancer fait migrer des chunks du shard ayant le plus de chunks vers celui qui en a le moins, jusqu'à ce que la collection devient équitablement répartie
- **Etapes :**
  - Le shard destination reçoit tous les documents du chunk à migrer
  - Le shard destination applique tous les changements faits aux données durant le processus de migration
  - Finalement, les méta-données concernant l'emplacement du chunk sur le config server sont modifier

Maintien d'une distribution équitable

- **Balancing**
- **Balacer:** processus en arrière plan, gérant les migrations de chunks
  - Peut être lancé à partir de n'importe quel Query Router
  - Quand la distribution des données est déséquilibrée, le balancer fait migrer des chunks du shard ayant le plus de chunks vers celui qui en a le moins, jusqu'à ce que la collection deviennent équitablement répartie



Ajout ou suppression de shards

- **Dans le cas d'un ajout de shard**
  - Un déséquilibre est crée entre les shards du cluster, car le nouveau shard n'a pas de chunks
  - MongoDB peut commencer le processus de migration immédiatement, mais cela risque de prendre du temps avant que le cluster ne soit équilibré
- **Dans le cas d'une suppression de shard**
  - Le balancer migre tous les chunks de ce shard vers les autres shards
  - Une fois toutes les données migrées et les méta-données mises à jour, la suppression peut avoir lieu

Stratégie de réplication

- **Définition d'un Replicat Set:** groupe de processus mongod (au niveau des workers) qui fournit la redondance et la haute disponibilité

- **Types de membres dans un réplica set:**
  - **Primaire:** reçoit toutes les opérations d'écriture (gérer les accès en écriture pour chaque copie)
  - **Secondaire:** réplication des opérations du primaire pour maintenir des répliques identiques à l'original
  - **Arbitre:** ne conserve pas de copie de la donnée, mais joue un rôle dans les élections qui sélectionnent un membre primaire, si le primaire actuel est indisponible

Stratégie de réplication

- **Primaire**
  - Seul membre qui reçoit les opération d'écriture
  - MongoDB applique les opérations d'écriture sur le primaire, puis les enregistre dans son **log (oplog)**
  - Les membres secondaires dupliquent ce log et appliquent les opérations sur leur données
  - Tous les membres d'un replicats set peuvent accepter une opération de lecture
    - Par défaut, une application dirige ces opérations vers le primaire
  - Un réplica set a au plus un primaire
  - Si le primaire tombe en panne, une élection a lieu pour en choisir un autre

Avantage:

consistance → on est sur d'avoir la toute dernière version modifié de la donnée

Stratégies de réplication

- **Secondaire**
  - Maintiennent une copie des données du primaire
  - Pour répliquer les données, un secondaire applique les opérations du oplog du primaire sur ses propres données, de manière asynchrone
    - *Oplog: les "operations log" est une collection Mongo spéciale qui contient les enregistrements tournants des données modifiées. A chaque modification d'une donnée (insertion, modification, suppression), une log est ajoutée avec un identifiant incrémenté.*
  - Un replicat set peut avoir un ou plusieurs secondaires
  - Même si les clients ne peuvent pas écrire des données sur les secondaires, ils peuvent en lire
  - des snapshots périodiques pour garder un historique

Stratégie de réplication

- **Arbitre:** gère les élections
  - N'a pas de copie des données et ne peut pas devenir un primaire
  - Permet de voter pour un primaire
  - Doit être défini pour les replicat sets avec un nombre pair de membres

Stratégie de réplication

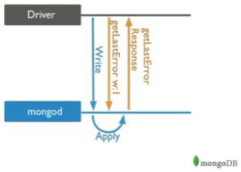
- **Elections**
  - Ont lieu à la création d'un replicat set, ou bien quand un primaire devient indispensable
  - Processus qui prend du temps, et qui rend le replicat set en **read only**
  - Chaque membre a une priorité déterminant son éligibilité à devenir primaire

Write concern

- Représente la garantie que MongoDB fournit en déclarant qu'une opération d 'écriture a été réalisée avec succès
- Si les insertions, modifications et suppressions ont un **write concern** faible
  - Les opérations retournent rapidement
  - En cas d'échec, les données peuvent ne pas être persistées
  - Si l'écriture a un **write concern** plus fort, les clients doivent attendre après chaque opération d'écriture que MongoDB la confirme
  - Depuis la version 2,6, le write concern peut être intégré avec l'opération d'écriture
    - Au lieu d'être une commande séparée

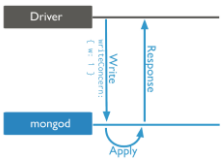
Write concern: Niveau 1

- **Unacknowledge:**
  - Similaire à « **Error ignored** »



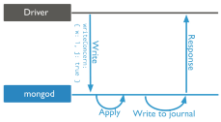
Write concern: Niveau 2

- **Acknowledge**
  - Niveau par defaut
  - Mongod confirme la réception de l'opération d'écriture et applique les changement à la **vue in-memory** des données (non journalisé)



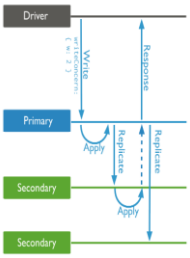
Write concern: Niveau 3

- **Journaled**
  - MongoDB valide les opérations d'écriture uniquement après avoir réalisé l'opération d'écriture sur le journal (Log permettant de sauvegarder les transaction et évènements)



Write concern: Niveau 4

- **Replica Acknowledged**
  - Garant que l'opération d'écriture s'est propagée à un nombre spécifié de réplicats

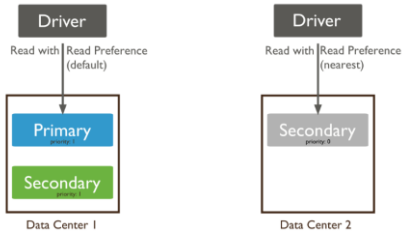


Read isolation

- **MongoDB** permet aux clients de lire des documents insérés ou modifiés avant qu'ils ne soient écrits sur le disque, indépendamment du write concern défini ou de la configuration de journalisation
- Comme résultat, les applications peuvent observer deux classes de comportement
  - Pour les systèmes avec des lectures et écrivains concurrents, MongoDB autorise les résultats de lecture avant que l'opération d'écriture ne retourne
  - Si mongoDB termine avant que la journalisation ne soit faite, et même si une opération d'écriture a retourné avec succès, le client peut avoir lu des données qui n'existeront plus quand mongod redémarrera

Read Preference

- Décrit comment les clients MongoDB routent les opérations de lecture vers les réplicas
- Par défaut, une application oriente ses opérations de lecture vers la données primaire
  - **Donne une garantie de fraîcheur, car les opérations d'écriture par défaut opèrent sur la donnée primaire**
- Pour les application qui ne nécessite pas des données de toute fraîcheur, il est possible d'améliorer la latence de lecture en distribuant certaines opérations de lecture vers les répliques


Driver
Read with Read Preference
(default)
Primary
Secondary
Data Center 1
Driver
Read with Read Preference
(nearest)
Secondary
Data Center 2